Лабораторна робота №6

Тема: Розробка консольного мережевого сніфера з використанням WinAPI та сокетів

Мета: Ознайомитись із можливостями WinAPI для роботи з мережевими інтерфейсами, навчитися використовувати низькорівневі сокети у режимі "promiscuous mode", отримувати та аналізувати мережеві пакети, розшифровувати заголовки протоколів Ethernet, IP, TCP/UDP.

1. Теоретичні відомості

1.1. Сирі сокети (Raw Sockets) та режим Promiscuous Mode

Мережевий сніфер це програмний засіб для перехоплення та побайтового аналізу пакетів у мережевому сегменті.

Для отримання сирого трафіку використовуються низькорівневі Raw-сокети (socket(AF_INET, SOCK_RAW, IPPROTO_IP)). На відміну від стандартних TCP/UDP сокетів, вони передають додатку пакети разом зі службовими заголовками безпосередньо від мережевого драйвера.

За замовчуванням мережева карта приймає лише пакети, адресовані безпосередньо їй (Unicast) або широкомовні (Broadcast). Для захоплення всього транзитного трафіку адаптер переводиться в режим проміскуїтету (Promiscuous Mode). У Windows це виконується викликом WSAIoctl із керуючим кодом SIO_RCVALL та прапорцем RCVALL_ON (вимагає запуску з правами Адміністратора).

1.2. Формати мережевих заголовків

Ethernet II (Layer 2, 14 байтів):

1. Destination MAC (6 Б) та Source MAC (6 Б) це фізичні адреси пристроїв;
2. EtherType (2 Б) це тип вкладеного протоколу (0x0800 для IPv4, 0x0806 для ARP).

IPv4 (Layer 3, мінімум 20 байтів):

1. Version / IHL (1 Б) це версія протоколу (4) та довжина заголовка (реальний розмір = IHL * 4 байти);
2. Total Length (2 Б) це повна довжина пакета;
3. TTL (1 Б) це час життя пакета (лічильник переходів між маршрутизаторами);
4. Protocol (1 Б) це код протоколу наступного рівня (1 - ICMP, 6 - TCP, 17 - UDP);
5. Source IP (4 Б) та Destination IP (4 Б) це адреси відправника й одержувача.

TCP (Layer 4, мінімум 20 байтів):

1. Source Port (2 Б) та Destination Port (2 Б) це порти з'єднання;
2. Sequence Number (4 Б) та Acknowledgment Number (4 Б) це порядкові номери байтів у потоці;
3. Data Offset / Flags (2 Б) це зсув даних і бітові прапорці керування сесією (SYN, ACK, FIN, RST, PSH).

UDP (Layer 4, фіксовані 8 байтів):

1. Source Port (2 Б) та Destination Port (2 Б);
2. Length (2 Б) це загальна довжина датаграми (заголовок + дані);
3. Checksum (2 Б) це контрольна сума.

1.3. Порядок байтів та вирівнювання структур

Network Byte Order (Big-Endian): усі багатобайтові числа в пакетах передаються старшим байтом уперед. В архітектурі x86/x64 використовується Little-Endian (молодший байт уперед). Для коректного читання портів і довжин обов'язково застосовуються функції конвертації:

1. ntohs() для 16-бітних чисел (порти, довжина);
2. ntohl() для 32-бітних чисел (поля Sequence/Ack у TCP).

Пакування пам'яті: щоб компілятор не додавав байти вирівнювання (padding), які зміщують поля структури, використовується директива #pragma pack(push, 1).

2. Алгоритм роботи програми

Ініціалізація підсистеми сокетів: виклик функції WSAStartup(MAKEWORD(2, 2), &wsa).

Отримання мережевих інтерфейсів хоста: виклики gethostname() та gethostbyname(). Отримані IP-адреси виводяться користувачеві для вибору активного адаптера.

Створення Raw-сокета: виклик socket(AF_INET, SOCK_RAW, IPPROTO_IP) (вимагає запуску від імені Адміністратора).

Прив'язка сокета (bind): занулення структури sockaddr_in через ZeroMemory(), внесення вибраної локальної IP-адреси та виклик bind().

Активація Promiscuous Mode: надсилання керуючого коду через WSAIoctl(sock, SIO_RCVALL, &flag, ...), де flag = RCVALL_ON.

Цикл перехоплення трафіку: блокуючий виклик recv() у буфер розміром 65536 байтів.

Побайтовий парсинг пакета:

перевірка наявності кадру Ethernet II;

накладання структури ip_header, перевірка версії IPv4 та обчислення довжини заголовка (IHL * 4);

визначення вкладеного протоколу (TCP, UDP або ICMP) та накладання відповідних структур транспортного рівня з конвертацією порядку байтів через ntohs() / ntohl();

вивід розпарсених полів у консоль у реальному часі.

Коректне завершення: перехоплення комбінації Ctrl+C через SetConsoleCtrlHandler, вимкнення режиму RCVALL_OFF, виклики closesocket() та WSACleanup().

3. Лістинг програмного коду

Лістинг файлу PacketParser.h:

```
#pragma once
#include "ProtocolHeaders.h"
#include <string>

namespace PacketParser {
    // Головний метод розбору буфера сирих байтів
```

```
    void ProcessPacket(const char* buffer, int size);

    // Допоміжні методи форматування адрес
    std::string FormatMac(const unsigned char* mac);
    std::string FormatIp(unsigned int ip);
    std::string FormatTcpFlags(unsigned short flags);
}
```

Лістинг файлу Sniffer.h:

```
#pragma once
#include <winsock2.h>
#include <ws2tcpip.h>
#include <mstcpip.h>
#include <vector>
#include <string>

namespace SnifferCore {
    // Ініціалізація Winsock2
    bool InitWinsock();

    // Отримання списку доступних локальних IPv4 адрес для прив'язки bind()
    std::vector<std::string> GetLocalIpAddresses();

    // Запуск прослуховування на обраній IP-адресі
    void StartSniffer(const std::string& bindIp);

    // Зупинка роботи та звільнення ресурсів
    void StopSniffer();
}
```

Лістинг файлу ProtocolHeaders.h:

```
#pragma once
#define _WINSOCK_DEPRECATED_NO_WARNINGS
#define _CRT_SECURE_NO_WARNINGS
#include <winsock2.h>
#include <windows.h>

#pragma pack(push, 1)

// 1. Заголовок Ethernet II (Layer 2) — 14 байт
struct eth_header {
    unsigned char dst[6];   // MAC-адреса призначення
    unsigned char src[6];   // MAC-адреса джерела
    unsigned short type;    // EtherType (0x0800 = IPv4, 0x0806 = ARP)
};

// 2. Заголовок IPv4 (Layer 3) — мінімум 20 байт
struct ip_header {
    unsigned char  ihl_version;   // 4 біти: версія (4), 4 біти: довжина заголовка (IHL)
    unsigned char  tos;           // Type of Service (DSCP/ECN)
    unsigned short total_length;  // Загальна довжина пакета (заголовок + корисне наван-
таження)
    unsigned short id;            // Ідентифікатор пакета для фрагментації
    unsigned short flags_offset;  // Прапорці (3 біти) та зсув фрагмента (13 біт)
    unsigned char  ttl;           // Час життя пакета (Time To Live)
    unsigned char  protocol;      // Протокол верхнього рівня (1 = ICMP, 6 = TCP, 17 =
UDP)
    unsigned short checksum;      // Контрольна сума IP-заголовка
    unsigned int   src;           // IPv4-адреса джерела (32 біти)
    unsigned int   dst;           // IPv4-адреса призначення (32 біти)
};

// 3. Заголовок TCP (Layer 4) — мінімум 20 байт
struct tcp_header {
```

```
    unsigned short src_port;      // Порт відправника
    unsigned short dst_port;      // Порт одержувача
    unsigned int   seq;           // Порядковий номер (Sequence Number)
    unsigned int   ack;           // Номер підтвердження (Acknowledgment Number)
    unsigned short offset_flags;  // 4 біти Data Offset + 6 бітів Reserved + 6 бітів
Flags
    unsigned short window;        // Розмір вікна прийому
    unsigned short checksum;      // Контрольна сума TCP
    unsigned short urgent_ptr;    // Вказівник терміновості (Urgent Pointer)
};

// 4. Заголовок UDP (Layer 4) - фіксовані 8 байт
struct udp_header {
    unsigned short src_port;      // Порт відправника
    unsigned short dst_port;      // Порт одержувача
    unsigned short length;        // Довжина UDP заголовка + даних
    unsigned short checksum;      // Контрольна сума UDP
};

// 5. Заголовок ICMP (Layer 3/4) - 8 байт
struct icmp_header {
    unsigned char  type;          // Тип повідомлення (0 = Echo Reply, 8 = Echo Request)
    unsigned char  code;          // Код типу
    unsigned short checksum;      // Контрольна сума ICMP
    unsigned short id;            // Ідентифікатор сесії
    unsigned short seq;           // Номер послідовності
};

#pragma pack(pop)
```

Лістинг файлу PacketParser.cpp:

```
#include "PacketParser.h"
#include <iostream>
#include <iomanip>
#include <sstream>

namespace PacketParser {

    std::string FormatMac(const unsigned char* mac) {
        std::stringstream ss;
        for (int i = 0; i < 6; ++i) {
            ss << std::hex << std::setw(2) << std::setfill('0') << std::uppercase <<
(int)mac[i];
            if (i < 5) ss << ":";
        }
        return ss.str();
    }

    std::string FormatIp(unsigned int ip) {
        struct in_addr addr;
        addr.s_addr = ip;
        return std::string(inet_ntoa(addr));
    }

    std::string FormatTcpFlags(unsigned short flags) {
        std::string res = "";
        // Прапорці знаходяться у молодших 6 бітах поля offset_flags
        if (flags & 0x0001) res += "FIN ";
        if (flags & 0x0002) res += "SYN ";
        if (flags & 0x0004) res += "RST ";
        if (flags & 0x0008) res += "PSH ";
        if (flags & 0x0010) res += "ACK ";
        if (flags & 0x0020) res += "URG ";
```

```
        return res.empty() ? "[NONE]" : res;
    }

    void ProcessPacket(const char* buffer, int size) {
        if (size < (int)sizeof(ip_header)) return;

        int ipOffset = 0;

        // Перевіряємо, чи буфер починається з Ethernet-заголовка (якщо кадр передано
повністю),
        // чи відразу з IP-заголовка (стандартна поведінка SIO_RCVALL у Windows).
        unsigned char firstByte = (unsigned char)buffer[0];

        if (size >= 14 && (unsigned char)buffer[12] == 0x08 && (unsigned char)buffer[13]
== 0x00) {
            // Маємо повний Ethernet II кадр (14 байт)
            const eth_header* eth = reinterpret_cast<const eth_header*>(buffer);
            std::cout << "[ETH]  SRC MAC: " << FormatMac(eth->src)
                << " -> DST MAC: " << FormatMac(eth->dst)
                << " | Type: 0x" << std::hex << std::setw(4) << std::setfill('0') <<
ntohs(eth->type) << std::dec << "\n";
            ipOffset = 14;
        }

        // Парсимо IP-заголовок
        const ip_header* ip = reinterpret_cast<const ip_header*>(buffer + ipOffset);

        // Перевіряємо версію IPv4 (старші 4 біти поля ihl_version мають дорівнювати 4)
        unsigned char ipVersion = (ip->ihl_version >> 4) & 0x0F;
        if (ipVersion != 4) {
            return; // Пропускаємо пакети, які не є IPv4
        }

        // Обчислюємо реальну довжину IP-заголовка: IHL * 4 байти (мінімум 20 байт)
        unsigned int ipHeaderLen = (ip->ihl_version & 0x0F) * 4;
        unsigned short totalLen = ntohs(ip->total_length);

        std::string protoStr = "UNKNOWN";
        if (ip->protocol == 1)  protoStr = "ICMP";
        else if (ip->protocol == 6)  protoStr = "TCP";
        else if (ip->protocol == 17) protoStr = "UDP";

        std::cout << "[IP]   SRC: " << FormatIp(ip->src)
            << " -> DST: " << FormatIp(ip->dst)
            << " | Proto: " << protoStr
            << " | TTL: " << (int)ip->ttl
            << " | Len: " << totalLen << " B\n";

        // Зсув до заголовка транспортного рівня (L4)
        int l4Offset = ipOffset + ipHeaderLen;

        // 1. Якщо це TCP
        if (ip->protocol == 6 && size >= (l4Offset + (int)sizeof(tcp_header))) {
            const tcp_header* tcp = reinterpret_cast<const tcp_header*>(buffer +
l4Offset);

            unsigned short srcPort = ntohs(tcp->src_port);
            unsigned short dstPort = ntohs(tcp->dst_port);
            unsigned short flags = ntohs(tcp->offset_flags) & 0x003F; // Молодші 6 біт
            unsigned int seq = ntohl(tcp->seq);

            std::cout << "[TCP]  " << srcPort << " -> " << dstPort
                << " | Flags: " << FormatTcpFlags(flags)
                << " | SEQ: " << seq << "\n";
        }
```

```
        // 2. Якщо це UDP
        else if (ip->protocol == 17 && size >= (l4Offset + (int)sizeof(udp_header))) {
            const udp_header* udp = reinterpret_cast<const udp_header*>(buffer +
l4Offset);

            unsigned short srcPort = ntohs(udp->src_port);
            unsigned short dstPort = ntohs(udp->dst_port);
            unsigned short length = ntohs(udp->length);

            std::cout << "[UDP]  " << srcPort << " -> " << dstPort
                << " | Length: " << length << " B\n";
        }
        // 3. Якщо це ICMP
        else if (ip->protocol == 1 && size >= (l4Offset + (int)sizeof(icmp_header))) {
            const icmp_header* icmp = reinterpret_cast<const icmp_header*>(buffer +
l4Offset);
            std::cout << "[ICMP] Type: " << (int)icmp->type
                << " | Code: " << (int)icmp->code << "\n";
        }

        std::cout << "-----------------------------------------------------------------
---\n";
    }
}
```

Лістинг файлу Sniffer.cpp:

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS
#include "Sniffer.h"
#include "PacketParser.h"
#include <iostream>

#pragma comment(lib, "ws2_32.lib")

namespace SnifferCore {
    static SOCKET g_rawSocket = INVALID_SOCKET;
    static bool g_isRunning = false;

    bool InitWinsock() {
        WSADATA wsa;
        // КРОК 1: Ініціалізуємо версію Winsock 2.2
        int res = WSAStartup(MAKEWORD(2, 2), &wsa);
        if (res != 0) {
            std::cerr << "[ПОМИЛКА] WSAStartup завершився з кодом: " << res << "\n";
            return false;
        }
        return true;
    }

    std::vector<std::string> GetLocalIpAddresses() {
        std::vector<std::string> ips;
        char hostName[256];

        if (gethostname(hostName, sizeof(hostName)) == SOCKET_ERROR) {
            return ips;
        }

        // КРОК 2: Отримуємо структуру хоста з IP-адресами інтерфейсів
        struct hostent* host = gethostbyname(hostName);
        if (host == nullptr) return ips;

        for (int i = 0; host->h_addr_list[i] != nullptr; ++i) {
            struct in_addr addr;
            memcpy(&addr, host->h_addr_list[i], sizeof(struct in_addr));
            ips.push_back(inet_ntoa(addr));
        }
```

```
        return ips;
    }

    void StartSniffer(const std::string& bindIp) {
        // КРОК 3: Створення сирого сокета (Raw Socket) для сімейства AF_INET
        g_rawSocket = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
        if (g_rawSocket == INVALID_SOCKET) {
            int err = WSAGetLastError();
            std::cerr << "[ПОМИЛКА] Не вдалося створити сирий сокет! Код помилки: " <<
err << "\n";
            if (err == WSAEACCES) {
                std::cerr << ">> ПРИЧИНА: Створення сирих сокетів вимагає прав
АДМІНІСТРАТОРА!\n";
            }
            return;
        }

        // КРОК 4: Прив'язка сокета (bind) до обраної локальної IP-адреси
        sockaddr_in localAddr;
        ZeroMemory(&localAddr, sizeof(localAddr));

        localAddr.sin_family = AF_INET;
        localAddr.sin_port = htons(0);
        localAddr.sin_addr.s_addr = inet_addr(bindIp.c_str());

        if (bind(g_rawSocket, (struct sockaddr*)&localAddr, sizeof(localAddr)) == SOCK-
ET_ERROR) {
            std::cerr << "[ПОМИЛКА] Помилка прив'язки bind() до адреси " << bindIp << ".
Код: " << WSAGetLastError() << "\n";
            closesocket(g_rawSocket);
            return;
        }

        // КРОК 5: Переведення адаптера у режим перехоплення всіх пакетів (Promiscuous
Mode)
        // Використовуємо WSAIoctl із керуючим кодом SIO_RCVALL
        DWORD flag = RCVALL_ON; // 1 = перехоплювати всі пакети (SIO_RCVALL)
        DWORD bytesReturned = 0;

        if (WSAIoctl(g_rawSocket, SIO_RCVALL, &flag, sizeof(flag), NULL, 0,
&bytesReturned, NULL, NULL) == SOCKET_ERROR) {
            std::cerr << "[ПОМИЛКА] WSAIoctl(SIO_RCVALL) завершився з помилкою: " << WSA-
GetLastError() << "\n";
            closesocket(g_rawSocket);
            return;
        }

        std::cout << "[УСПІХ] Проміскуїті-режим активовано на інтерфейсі " << bindIp <<
".\n";
        std::cout << "[ІНФО] Початок перехоплення трафіку (натисніть Ctrl+C для
зупинки)...\n";
        std::cout <<
"=====================================================================\n";

        g_isRunning = true;
        char packetBuffer[65536]; // Максимальний можливий розмір IPv4 пакета (64 КБ)

        // КРОК 6: Цикл безперервного прийому пакетів
        while (g_isRunning) {
            int bytesReceived = recv(g_rawSocket, packetBuffer, sizeof(packetBuffer), 0);

            if (bytesReceived > 0) {
                // КРОК 7: Розбір та вивід кожного отриманого пакета
                PacketParser::ProcessPacket(packetBuffer, bytesReceived);
            }
```

```
            else if (bytesReceived == SOCKET_ERROR) {
                if (g_isRunning) {
                    std::cerr << "[ЗБІЙ] recv() повернув помилку: " << WSAGetLastError()
<< "\n";
                }
                break;
            }
        }
    }

    void StopSniffer() {
        g_isRunning = false;
        if (g_rawSocket != INVALID_SOCKET) {
            // КРОК 8: Вимикаємо проміскуїті-режим та закриваємо сокет
            DWORD flag = RCVALL_OFF;
            DWORD bytesReturned = 0;
            WSAIoctl(g_rawSocket, SIO_RCVALL, &flag, sizeof(flag), NULL, 0,
&bytesReturned, NULL, NULL);

            closesocket(g_rawSocket);
            g_rawSocket = INVALID_SOCKET;
        }
        WSACleanup();
        std::cout << "\n[ІНФО] Роботу сніфера коректно зупинено. Ресурси звільнено.\n";
    }
}
```

Лістинг файлу main.cpp:

```
#include "Sniffer.h"
#include <iostream>
#include <io.h>
#include <fcntl.h>

// Обробник сигналу Ctrl+C для коректного закриття сокета
BOOL WINAPI ConsoleCtrlHandler(DWORD ctrlType) {
    if (ctrlType == CTRL_C_EVENT || ctrlType == CTRL_CLOSE_EVENT) {
        SnifferCore::StopSniffer();
        return TRUE;
    }
    return FALSE;
}

int main() {
    system("chcp 65001 > nul");
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    std::cout << "     МЕРЕЖЕВИЙ СНІФЕР ПАКЕТІВ WINSOCK2 RAW SOCKETS (ЛАБ. №6)\n";

    // Реєструємо перехоплювач натискання Ctrl+C
    SetConsoleCtrlHandler(ConsoleCtrlHandler, TRUE);

    if (!SnifferCore::InitWinsock()) {
        return 1;
    }

    // Отримуємо список локальних інтерфейсів
    auto ips = SnifferCore::GetLocalIpAddresses();
    if (ips.empty()) {
        std::cerr << "[ПОМИЛКА] Не знайдено жодного активного мережевого IPv4 інтерфей-
су!\n";
        WSACleanup();
        return 1;
    }
```

```
    std::cout << "Знайдені мережеві інтерфейси хоста:\n";
    for (size_t i = 0; i < ips.size(); ++i) {
        std::cout << " [" << i + 1 << "] " << ips[i] << "\n";
    }

    std::cout << "\nОберіть номер інтерфейсу для прослуховування (1-" << ips.size() <<
"): ";
    int choice = 1;
    std::cin >> choice;

    if (choice < 1 || choice >(int)ips.size()) {
        choice = 1;
    }

    std::string selectedIp = ips[choice - 1];
    std::cout << "Обрано інтерфейс: " << selectedIp << "\n\n";

    // Запускаємо перехоплення (потрібні права Адміністратора!)
    SnifferCore::StartSniffer(selectedIp);

    SnifferCore::StopSniffer();
    return 0;
}
```

4. Результати виконання програми

Під час тестування утиліти на робочій станції було ініціалізовано перехоплення трафіку на фізичному адаптері Wi-Fi (192.168.0.43).

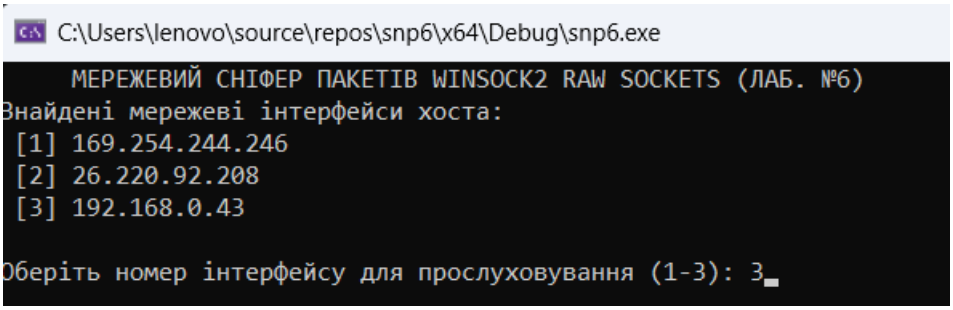


Рис. 1. Меню вибору активного мережевого інтерфейсу в консолі застосунку.

Після активації режиму SIO_RCVALL утиліта розпочала побайтовий розбір вхідного та вихідного трафіку в реальному часі.

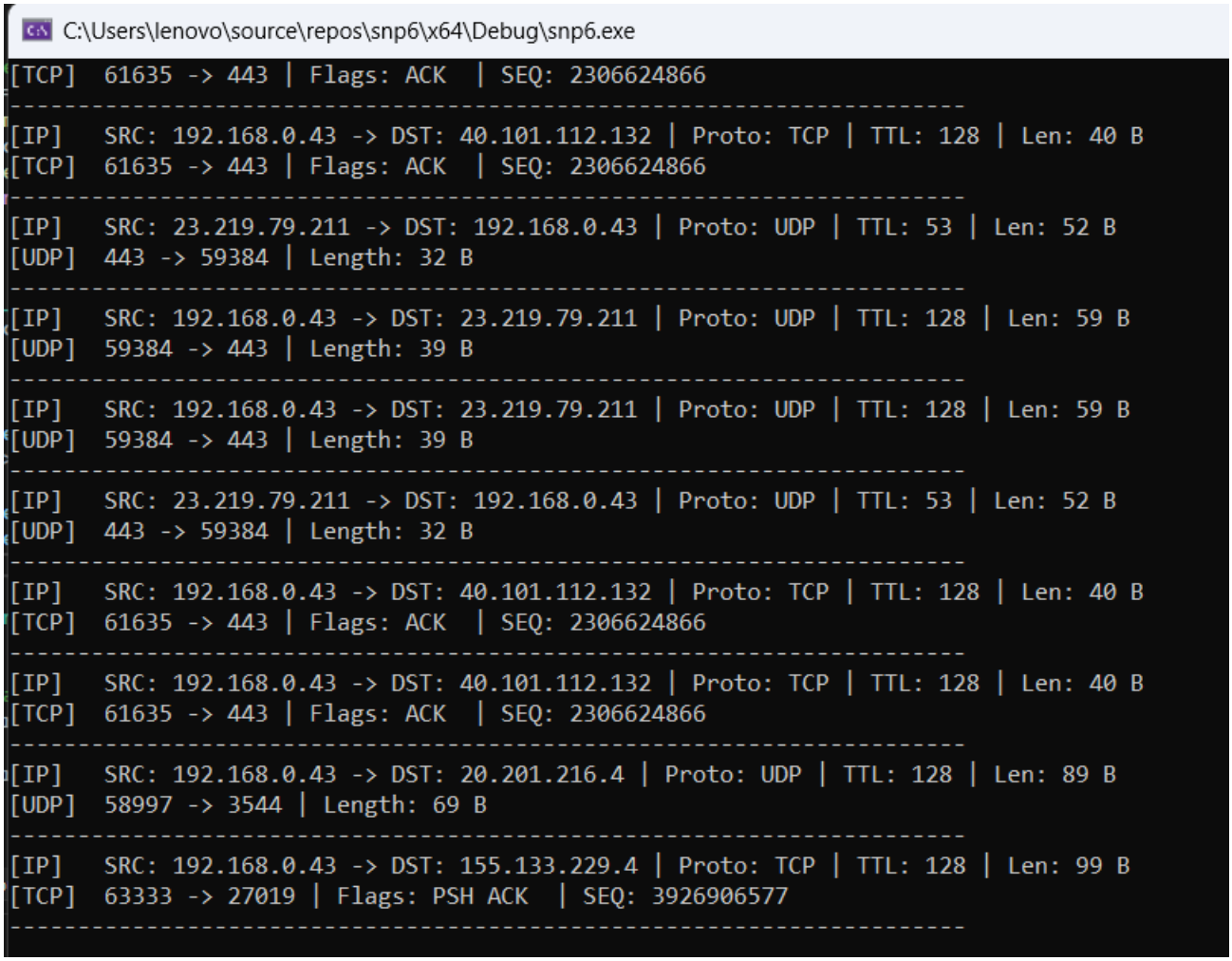


Рис. 2. Вікно консолі з результатами реального перехоплення та декодування пакетів протоколів IPv4, UDP та TCP.

Аналіз отриманих результатів:

Двосторонній обмін (Ingress / Egress): сніфер успішно перехоплює як вихідні пакети локального вузла (SRC: 192.168.0.43), так і вхідні відповіді з інтернету.

Аналіз поля TTL: для вихідних пакетів із Windows значення TTL дорівнює 128 (системне значення). Для вхідних пакетів із зовнішньої мережі CDN Akamai (SRC: 23.219.79.211) значення TTL становить 53, що свідчить про проходження пакета через приблизно 11 проміжних маршрутизаторів.

Ідентифікація протоколів L4:

Протокол UDP (порт 443): перехоплено датаграми протоколу QUIC / HTTP/3 (сесія з 23.219.79.211), а також службовий трафік Windows на порт 3544 (Teredo tunneling до сервера Microsoft 20.201.216.4).

Протокол TCP: зафіксовано HTTPS-з'єднання з сервером Microsoft (40.101.112.132:443) із прапорцем підтвердження ACK, а також ігровий трафік платформи Steam / Valve (155.133.229.4:27019) із комбінованими прапорцями PSH ACK для негайної передачі даних прикладному рівню.

Висновок: Я ознайомився із можливостями WinAPI для роботи з мережевими інтерфейсами, навчився використовувати низькорівневі сокети у режимі "promiscuous mode", отримувати та аналізувати мережеві пакети, розшифровувати заголовки протоколів Ethernet, IP, TCP/UDP.

GIT: https://github.com/dmitro413/snp6